

Fouille de Données Massives TP – Adaptation de Domaine

M2 Informatique - SISE (2026-2027)

Guillaume Metzler

Institut de Communication (ICOM)

Université de Lyon, Université Lumière Lyon 2

Laboratoire ERIC UR 3083, Lyon, France

`guillaume.metzler@univ-lyon2.fr`

Résumé. Ce TP met en pratique les méthodes de repondération d’instances vues en cours (séance 6) : après un rappel guidé sur l’exemple filé de régression 1D, vous construirez et comparerez quatre méthodes d’estimation du ratio de densités (*NN weighting*, KDE, classifieur de domaine/IWC, KMM) sur un problème de classification 2D conçu pour ce TP, avant de les appliquer à un jeu de données réel sous biais de sélection. Le TP s’appuie sur le notebook `tp_da.ipynb` (à compléter) et les modules `utils.py`, `toydatasets.py` (déjà vus en cours) et `tp_utils.py` (nouveau). La correction complète et exécutée est fournie dans `tp_da_solutions.ipynb`.

1 Introduction

L’objectif de ce TP est double : consolider la mécanique de la repondération d’instances sur un exemple simple et entièrement analytique (comme dans le cours), puis mesurer, sur des données simulées et réelles, l’écart de performance entre un modèle « source seule » et un modèle corrigé par adaptation de domaine.

Récupérez l’archive de ressources associée à ce TP, qui contient :

- `tp_da.ipynb` : le notebook à compléter (questions numérotées, cellules de code vides à remplir) ;
- `utils.py`, `toydatasets.py` : déjà utilisés en cours (fonctions `gaussian`, `show_gaussian`) ;
- `tp_utils.py` : nouveau module pour ce TP, contenant la génération de données avec covariate shift (`generate_shift_classification`) et les quatre méthodes de repondération (`nn_weights`, `kde_weights`, `iwc_weights`, `kmm_weights`), ainsi que le calcul de l’*Effective Sample Size* (`effective_sample_size`).

Remarque Toutes les questions de ce TP demandent d’écrire du code dans le notebook `tp_da.ipynb`. Ce document décrit les questions et les attendus ; il ne remplace pas le notebook, qu’il faut garder ouvert en parallèle.

2 Partie 1 – Rappel guidé : régression pondérée (30 min)

On reprend l’exemple filé du cours : $P_S(X) = \mathcal{N}(-1, 1)$, $P_T(X) = \mathcal{N}(1, 1)$, et $P_S(Y | X = x) = P_T(Y | X = x) = \mathcal{N}(|x|, 0.25)$ (covariate shift). On dispose d’un échantillon source labellisé de taille $m = 30$ et d’un échantillon cible **non labellisé** de taille $n = 30$.

1. Ajustez un modèle linéaire sur la source seule et affichez-le (Question 1.1 du notebook).
2. Calculez ses erreurs quadratiques moyennes sur source et sur cible ; commentez l'écart (Question 1.2).
3. Implémentez le ratio de densités exact $w(x) = \exp(2x)$ (rappelé en cours) et affichez-le superposé aux deux densités (Question 1.3).
4. Ajustez un modèle repondéré (`sample_weight=w`) et comparez son erreur cible à celle du modèle source seul (Question 1.4). L'amélioration observée est-elle cohérente avec la Proposition 3.4 du cours ?

3 Partie 2 – Classification 2D sous covariate shift (1h15)

C'est le cœur du TP. On travaille sur un problème de classification binaire construit par `generate_shift_classification` : la vraie règle de décision est $y = +1$ si x_1 appartient à une bande $[-0.5, 1.8]$ (et -1 sinon) – une règle **non linéaire** en x_1 , volontairement mal spécifiée par un modèle linéaire, exactement comme la fonction $|x|$ de la Partie 1. La source est concentrée près de $x_1 = 0$ (elle voit surtout l'intérieur de la bande), la cible est concentrée bien plus loin (surtout à l'extérieur, côté droit) : c'est un cas de covariate shift sévère.

1. Générez le jeu de données (`seed=2`) et visualisez-le : source/cible, classes, bande vraie (déjà fourni dans le notebook – prenez le temps de bien comprendre la figure avant de continuer).
2. **Baseline.** Entraînez une régression logistique sur la source seule ; mesurez son accuracy sur la cible. Le résultat est-il cohérent avec la construction du jeu de données (Question 2.1) ?
3. **Les quatre méthodes.** Calculez les poids `w_nn`, `w_kde`, `w_iwc`, `w_kmm` à l'aide de `tp_utils.py`, ré-entraînez un modèle pondéré pour chacune, et mesurez leur accuracy cible (Question 2.2).
4. **Comparaison visuelle.** Représentez les cinq accuracies (source seule + 4 méthodes) sur un même diagramme en barres, avec une ligne horizontale représentant un modèle *oracle* entraîné directement sur les données cible labellisées (borne haute théorique, non disponible en pratique) (Question 2.3).
5. **Effective Sample Size.** Calculez et commentez l'ESS de chaque méthode de repondération, en la comparant à $m = 250$. Quelle méthode répartit le mieux le poids sur l'échantillon source ? Faites le lien avec l'Exercice 1.3 du TD (Question 2.4).
6. **Visualisation du classifieur de domaine.** Affichez la carte de probabilité $\phi(x) = \hat{\mathbb{P}}(\text{cible} \mid x)$ du classifieur de domaine utilisé pour l'IWC, avec les points source et cible superposés (Question 2.5).

Piège classique Pensez à ne **jamais** utiliser `yt` pour entraîner un modèle – uniquement pour évaluer. C'est une règle absolue de l'adaptation de domaine non supervisée : dans un cas réel, ces étiquettes n'existent tout simplement pas.

4 Partie 3 – Application sur données réelles (30 min)

On simule un biais de sélection d'échantillon sur le jeu `breast_cancer` de `scikit-learn` : le domaine *source* est construit en sur-représentant les tumeurs de petit rayon (`mean radius` faible), le domaine *cible* regroupe le reste des observations.

1. Le code de génération de ce jeu de données biaisé est fourni dans le notebook : prenez le temps d'observer la différence de proportion de la classe positive entre source et cible.
2. Entraînez un modèle sur la source seule, évaluez-le sur la cible. Calculez les poids IWC, ré-entraînez un modèle repondéré, comparez les deux accuracies cible ainsi que l'ESS obtenue (Question 3.1).
3. Comparez le gain obtenu ici à celui observé en Partie 2. Est-il du même ordre de grandeur ? Proposez une explication.

5 Partie 4 – Pour aller plus loin (bonus, non noté)

Cette partie est ouverte, à traiter selon votre temps disponible : subspace alignment (Fernando et al., 2013), étude de sensibilité aux hyperparamètres des méthodes de repondération, esquisse d'un DANN simplifié sous `pytorch`, ou variation du scénario de biais de sélection de la Partie 3. Le détail des pistes proposées se trouve directement dans le notebook.